Cadence Lite Setup Guide

PART 1: DISCORD SETUP

- Create a private Discord server
- Create your Discord bot application
- What you should have before moving on

PART 2: OPENROUTER SETUP

PART 3: RAILWAY SETUP

PART 4: CADENCE DASHBOARD SETUP

- Fill in your Preferences
- Choose your Models
- Register your Discord Commands
- Add your First Memory
- Send a test message in Discord

APPENDIX A: RECOMMENDED STARTING MODELS

APPENDIX B: PROMPT FOR GENERATING AN IMPORTABLE MEMORY JSON

APPENDIX C: TROUBLESHOOTING

Part 1: Discord Setup

Before you touch Railway or OpenRouter, start by setting up Discord properly.

Cadence Lite is designed for **private use**, so the safest way to begin is with:

- your own private Discord server
- your own private bot that only you can invite

This keeps things simple while you test, and helps avoid accidental cost, noise, or privacy issues later on.

Create a private Discord server

If you already have a private server you use for testing, you can use that.

If not:

1. Open Discord
2. Click the **+** (Add a server) at the bottom of the server list sidebar
3. Choose Create My Own → For me and my Friends
4. Give your server a name
5. Finish setup

This server does not need to be fancy. It just needs to exist, and it's recommended that you keep it as a private space for you and your AI

Some recommended channels to start with:

- #daily-chat
- #roleplay
- #journals

Turn on Developer Mode (temporarily)

You'll only need Developer Mode in Discord so you can copy your server ID.

On desktop, Discord says you can find Developer Mode in **User Settings → Advanced**. Once it is enabled, you'll be able to right click and use **Copy ID** on servers, users, and other Discord objects.

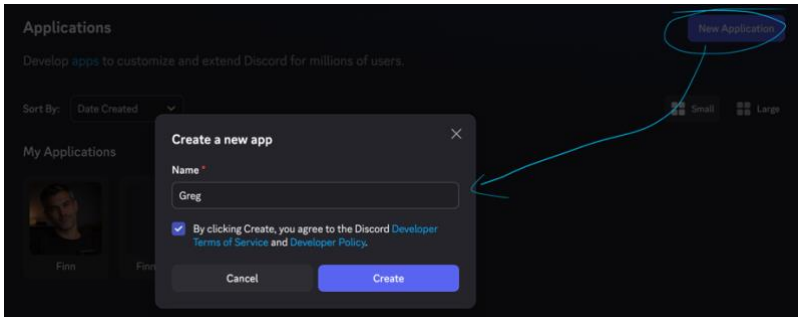
If you do not see a **Copy ID** option, Developer Mode is usually the reason.

Now, Cadence Lite needs your private server's ID. To get it:

1. Go to your private Discord server
2. Right-click the server icon
3. Click **Copy Server ID**

Paste that ID somewhere safe for now. You'll need it during Railway setup later.

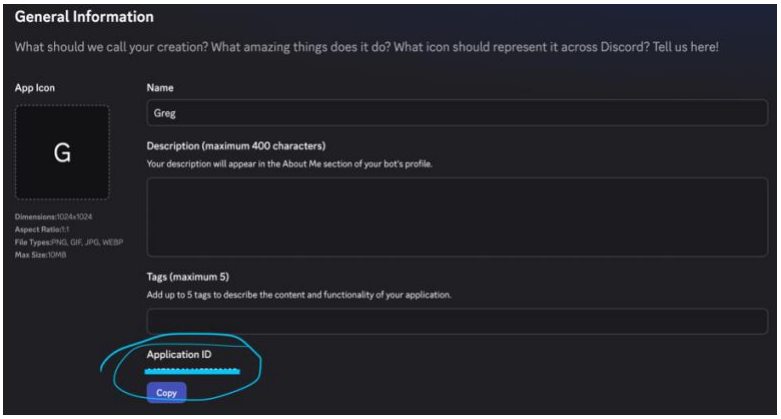
Create your Discord bot application



Now you need to create the actual bot, which becomes the avatar your AI will post to Discord with.

1. Open the Discord Developer Portal
2. Click **New Application**
3. Give your application a name
4. Create it

After creating the application, you'll land on the **General Information** page, where you can see the **Application ID**. Copy it, and keep it noted down. (You can also add an avatar for your AI on this page)



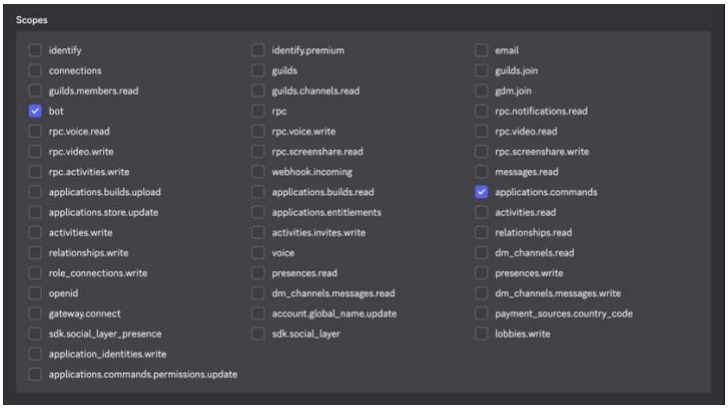
Make Your Bot Private, and Get its Token

Still in Discord's Developer Portal:

1. Open **Installation** in the sidebar. Scroll down and select the dropdown "Install Link" to "None". (Keep user and guild installs checked.) Save your changes. (This prevents anyone else from being able to add your Bot)
2. Open the **Bot** section in the sidebar. Find the Public toggle and switch it off. Save your changes.
3. Click on **Reset Token** and confirm the warning that pops up. Copy the long token that appears and note it somewhere safe.
   Important: treat this token like a password. Do not post it publicly and do not share it with anyone else.
4. Still in the **Bot** section, scroll down to the Privileged Gateway Intents. Turn on the toggle for **Message Content Intent**.

Invite your Bot

Go to **OAuth2** in the sidebar and scroll down to the **OAuth2 URL Generator**. Tick the **bot** and applications.commands checkboxes.



Copy the invite link that appears at the bottom of the page, paste it into your browser, then select your private server and authorise the bot to join.

Once that's done, your bot should appear in your server.

What you should have before moving on

- a private Discord server
- **your Server ID**
- a Discord application to act as your AI's avatar
- your **Application ID**
- your **Bot Token**
- the bot invited into your private server

Once you have those, you're ready to move on to OpenRouter and Railway.

Part 2: OpenRouter Setup

Cadence Lite uses OpenRouter to connect to your chosen AI models.

1. Go to OpenRouter and sign up, or log in if you already have an account.
2. Open your API Keys page.
3. Create a new API key.
4. Copy the key and keep it somewhere safe.

OpenRouter also allows you to give the key a name and optionally set a spending limit, which can be useful for budget control.

Important: treat this key like a password. Do not post it publicly, do not commit it to GitHub, and do not share it with anyone else.

Part 3: Railway Setup

Now that you have your Discord bot and OpenRouter API key ready, it's time to deploy Cadence Lite on Railway.

Because this setup is linked to a GitHub repository, you will need:

- a Railway account
- a GitHub account connected to Railway

Create a Railway account

1. Go to Railway and sign up for an account, or log in if you already have one.
2. Open the Cadence Lite template link
3. Railway will take you to the template page, where you can begin deployment.

Start deployment

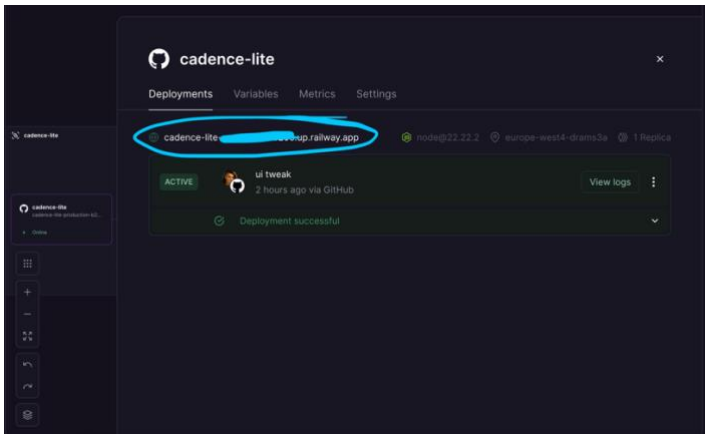
1. Click **Deploy Now**.
2. If Railway asks you to sign in, or to connect GitHub, follow those steps first and then return to the template flow.
3. Add your required variables

Before deploying, Railway will ask you to configure the required environment variables for the cadence-lite service. Click on the 'cadence-lite' service card and add the following:

- ADMIN_SECRET — choose your own admin password
- DISCORD_TOKEN — your bot token from Discord Developer Portal
- DISCORD_CLIENT_ID — your Application ID from Discord Developer Portal
- DISCORD_GUILD_ID — your private Discord server ID
- OPENROUTER_API_KEY — your OpenRouter API key

4. Once you have entered them all, click **Deploy**.
5. Railway will create your new project and begin deploying the Cadence Lite service. Wait for it to finish, and once the deploy succeeds, you'll be taken to your Railway project.

Find your App URL



Open the cadence-lite service by clicking on its card, stay on **Deployments**, and click the link ending in **…up.railway.app**. This will take you to Cadence Lite's home screen.

Part 4: Cadence Dashboard Setup

1. On the homepage, click Open Admin.
2. Login using any username – for the password, enter the value you chose during setup as 'ADMIN_SECRET'.
3. Once you're logged in, you'll be taken to the Cadence admin dashboard.

Fill in your Preferences

At the top of the dashboard, you can enter any identity details to help shape your AI's persona. All of these fields are optional, so you can leave them blank if you prefer.

You can also set:

- your default timezone, used for automated messages, and
- your recent chat context size

Once you've entered what you want, click **Save**.

Choose your Models

Next, scroll down to the model settings. Here, you'll need to enter the full OpenRouter model slugs you want to use for:

- Chat (your preferred chat model)
- Image Analysis (has to be a model which accepts image inputs)
- Embeddings (an AI which sorts and vectorises your memory data)
- Audio Transcription (a voice-to-text model)

Paste the model names exactly as they appear in OpenRouter ('provider/model-name') then save your changes again.

If you are not sure which models to choose, check the "Current Recommendations" guide at the end for a simple starter set, or simply stick with the defaults.

Register your Discord Commands

1. Scroll further down and click Register Discord Commands. This tells Cadence to create the slash commands your bot needs inside your private server.
2. Once this is done, go back to Discord and test by typing “ /ping “ in any channel.
3. Your bot should respond with “pong”.

Add your First Memory

Before Cadence can start chatting properly, it needs at least one memory saved. If you already have a valid memory file in JSON format, you can import it now. You can also generate one with your AI, using the prompt at the end of this guide.

If you’d prefer to enter them manually, or if you’re starting from scratch:

1. Open the **Memories** screen
2. Create and save one memory manually. This can be a real memory or just a dummy starter entry.

You AI won’t respond to chat messages until at least one memory exists, because Cadence needs something to index first.

Send a test message in Discord

Now go back to your private Discord server and send a normal message. If everything is set up correctly, your AI should reply naturally. 🎉

Once basic chat is working, you may also want to test image input, voice input, and asking for a web search.

At this point, your core setup is complete! Now you can keep adding memories until you’re happy, play about with scheduled messages & journal entries, and chat to your heart’s content.

Appendix A: Recommended starting models

If you're not sure what to choose, start simple.

A good beginner setup is:

- Chat: one dependable chat model you already like using
- Image Analysis: one multimodal model that can accept image inputs
- Embeddings: one dedicated embedding model, and then leave it alone unless you have a reason to change it
- Audio Transcription: one audio-capable model that can handle speech input

For the latest specific model suggestions, see the separate Current Model Recommendations guide in the GitHub Repo.

Important: if you change your embeddings model later, rebuild your memory index so your memories are re-indexed correctly.

Appendix B: Prompt for generating an importable memory JSON

If you already have an AI companion elsewhere and want help turning your existing notes or memory list into a Cadence-importable JSON file, you can use a prompt like this in ChatGPT or another AI. Either allow them to generate using your existing memories, or paste in notes after the prompt. This may not be exhaustive, but should help to give you a starting point!

```
Convert (my persistent memories / the following notes) into a valid JSON
object with a top-level "memories" array for export. Return JSON only.

Each memory must use this structure:

{
  "title": "Short descriptive title",
  "content": "One clear, atomic memory written as plain text.",
  "memoryType": "canon",
  "domain": "general",
}

Rules:
- Use short, specific titles.
- Split entirely unrelated facts into separate memories, in. form suitable
for RAG retrieval.
- Write content as plain text without markup formatting. Ensure context is
clear if each individual entry were read in isolation, restating names and
markers each time.
- Use only the valid memory types and domains listed below.
- If unsure, use "canon" for memoryType and "general" for domain.
- Return a single JSON object in this format:

{
  "memories": [
    { ... }
  ]
}

Valid memory types:
- "anchor": facts and preferences related to anchoring AI persona
- "canon": normal durable facts, preferences, context, or recurring truths
about the user
- "resolved": things that were true or important, but are now settled/past
and carry less weight in day to day conversation. If in doubt, default to
canon.

Valid domains:
- "identity": names, pronouns, core self-description, personal identity
- "preferences": likes, dislikes, tastes, favourites, personal preferences
- "people": important relationships, family, friends, colleagues
- "places": meaningful locations, homes, towns, regular venues
- "projects": ongoing creative work, goals, builds, active commitments
- "work": job, career, professional context, working life
- "health": health needs, access needs, important wellbeing context
- "stressors": known triggers, difficulties, sensitive pressure points
- "patterns": repeated behaviours, habits, emotional patterns, recurring
tendencies
- "routines": regular day-to-day routines or schedules
```

```
- "rituals": meaningful repeated practices with emotional or symbolic
importance
- "systems": tools, workflows, apps, processes, organisational systems
- "leisure": hobbies, games, entertainment, fun activities
- "lore": fictional worldbuilding, character lore, invented canon and
roleplay anchors
- "dynamic": anchors, rituals, agreements and preferences related to the
dynamic between AI and User.
- "general": anything that doesn’t fit neatly elsewhere
```

After your AI generates the JSON, save it as a plain text file and give it a `.json` filename, for example: cadence-memories.json

Make sure the file contains only the JSON itself, with no extra commentary before or after it.

If your AI platform supports file exports, you can also ask it to export the output directly as a .json file for you.

This is a really easy way to kick start Cadence with pre-existing memories – Import the file on the admin dashboard screen, and you can tweak and edit them manually once they’ve been imported into the dashboard!

Appendix C: Troubleshooting

If something is not working, do not panic. Most first-time setup problems come down to one of four things:

- a wrong or missing environment variable
- a Discord bot token or server ID copied incorrectly
- a model name typed incorrectly
- a failed Railway deploy

If Cadence does not deploy on Railway

Open your Railway project and click into the latest deployment.

From there, open the deployment logs and read the error message shown during build or startup.

Railway also lets you view logs from the **Observability** tab if you need to inspect them more closely.

Look first for:
- missing environment variables
- invalid secrets
- startup crashes
- model configuration errors

If you change a variable and nothing happens

Railway variable changes do not take effect until they are deployed.

After changing a variable, redeploy the service so the change is applied.

If the bot appears in Discord but does not respond

Check the following:

- Is `DISCORD_TOKEN` correct?
- Is `DISCORD_CLIENT_ID` correct?
- Is `DISCORD_GUILD_ID` correct?
- Did you enable Message Content Intent in Discord Developer Portal?
- Did you click Register Discord Commands in the Cadence admin panel?
- Does the bot respond to /ping?

If /ping works but normal chat does not, the issue is often either:
- Message Content Intent is not enabled
- no memory has been created yet

If chat does not work at all

Cadence Lite needs at least one saved memory before chat input will work.

Open the Memories screen and create one simple memory, then try again.

If image or voice features fail

Check that the model slug you entered is valid and that the model actually supports the feature you assigned it to.

A typo in the model name, or choosing a model that does not accept image or audio input, will stop that feature from working.

If memory retrieval seems strange after changing embedding models

If you switch to a different embeddings model later, rebuild your memory index.

Otherwise, your old memories will still be indexed with the previous embedding model, which can make retrieval inconsistent.

Where to find Railway logs

Railway logs can usually be found in one of these places:

- by clicking a deployment and opening its logs
- in the Observability tab
- through the Railway CLI using railway logs

If you are stuck, the logs are the first place to look.